# 只算到第 **23** 层的 **prefill**：请求形状 × 并发（**H20** 单卡）

## 结论

1. 单个请求就基本把 **GPU** 占满了。 并发从 1 加到打满，吞吐只涨 13–20%，多出来的请求只是排队。所以延迟大致和并发数成正比：4k 请求每多一个并发约加 0.19 秒，64k 请求约加 8.5 秒。
2. **prefill size** 设 **4k** 还是用默认值（**chunk 8192**），**token** 吞吐几乎一样，差距在 1–2% 以内，两档的延迟曲线也基本重合。
3. 给定延迟下最多支持的并发 ≈ 延迟要求 × 吞吐 ÷ 每个请求的 **token** 数。 按 p90 算：4k 请求要求 1 秒内大约能撑 4 个并发；64k 请求要求 60 秒内只能撑 4 个。
4. 长请求在 **token** 层面更贵：打满时 4k 请求约 23.4k tok/s，64k 请求只有约 8.5k tok/s，每个 token 贵 2.75 倍。原因是 attention 的计算量随上下文长度增长。
5. 分 **4** 段时，复用 **KV cache** 能让单位时间服务的 **token** 数翻倍（约 20k 对 9.5k tok/s），p90 延迟也降到一半以下。但这要求 KV 池放得下所有在途轨迹：并发 32 条轨迹时，KV 需求约 200 万 token，超过 KV 池的 107 万容量，前面轮次的 KV 被逐出，缓存命中从 61% 掉到 29%，优势基本消失。

## 测试设置

| 项目 | 设置 |
|---|---|
| GPU / 模型 | 1 × H20（141 GB），Qwen3-8B-FP8 |
| 计算范围 | 只跑 **decoder block 0..23**（36 层的 2/3），输出第 23 层残差流；不做最终 norm、lm_head 和采样。模型按 24 层构建，所以权重（6.7 GB）和 KV 池也只按 24 层分配 |
| 推理框架 | SGLang 0.5.19 官方 wheel，进程内 Engine，真实调度器（continuous batching、chunked prefill、radix cache），FP8 GEMM 用 DeepGEMM |
| 请求 | 直接传 token ids，`max_new_tokens=1`：prefill 完即结束，没有 decode |
| KV 池 | 1,067,688 token（`mem_fraction_static` 0.75） |
| prefill size | **4k**：`chunked_prefill_size` 和 `max_prefill_tokens` 都设 4096；默认：SGLang 按显存自动选，即 chunk 8192、`max_prefill_tokens` 16384 |
| 负载 | 闭环：始终有 C 个请求（分段形状为 C 条轨迹）在途。每档至少跑 60 秒，且至少完成 4×C 个请求 |
| 指标 | 计算 **tok/s** = 实际做 prefill 的 token（输入减去缓存命中）÷ 时间；服务 **tok/s** = 全部输入 token ÷ 时间；延迟 = 客户端从提交到完成（含排队） |

数据：从 WildChat 抽 1 万条 3.5k–4.5k token 的对话（平均 3,968 token），从 SWE-rebench OpenHands 轨迹抽 1 万条 56k–72k token 的（平均约 62.6k token）。这两批只作为样本池，每档按固定种子抽取，两档 prefill size 用的是同一串请求。

四种请求形状：

| 形状 | 内容 | **radix cache** |
|---|---|---|
| short | 独立的 ~4k 请求 | 关 |
| long | 独立的 ~64k 请求，一次性 prefill | 关 |
| long_seg_cache | 一条长轨迹在最接近 1/4、2/4、3/4 长度的 assistant 轮次边界切开，变成 4 个累积前缀请求（约 16k / 32k / 48k / 64k），按顺序发送；后一段复用前一段的 KV | 开 |
| long_seg_nocache | 同样的轨迹、同样的 4 段，但前面轮次的 KV 已经被逐出，每段都要重算整个前缀 | 关 |

## **1.** 吞吐和延迟：单请求就打满

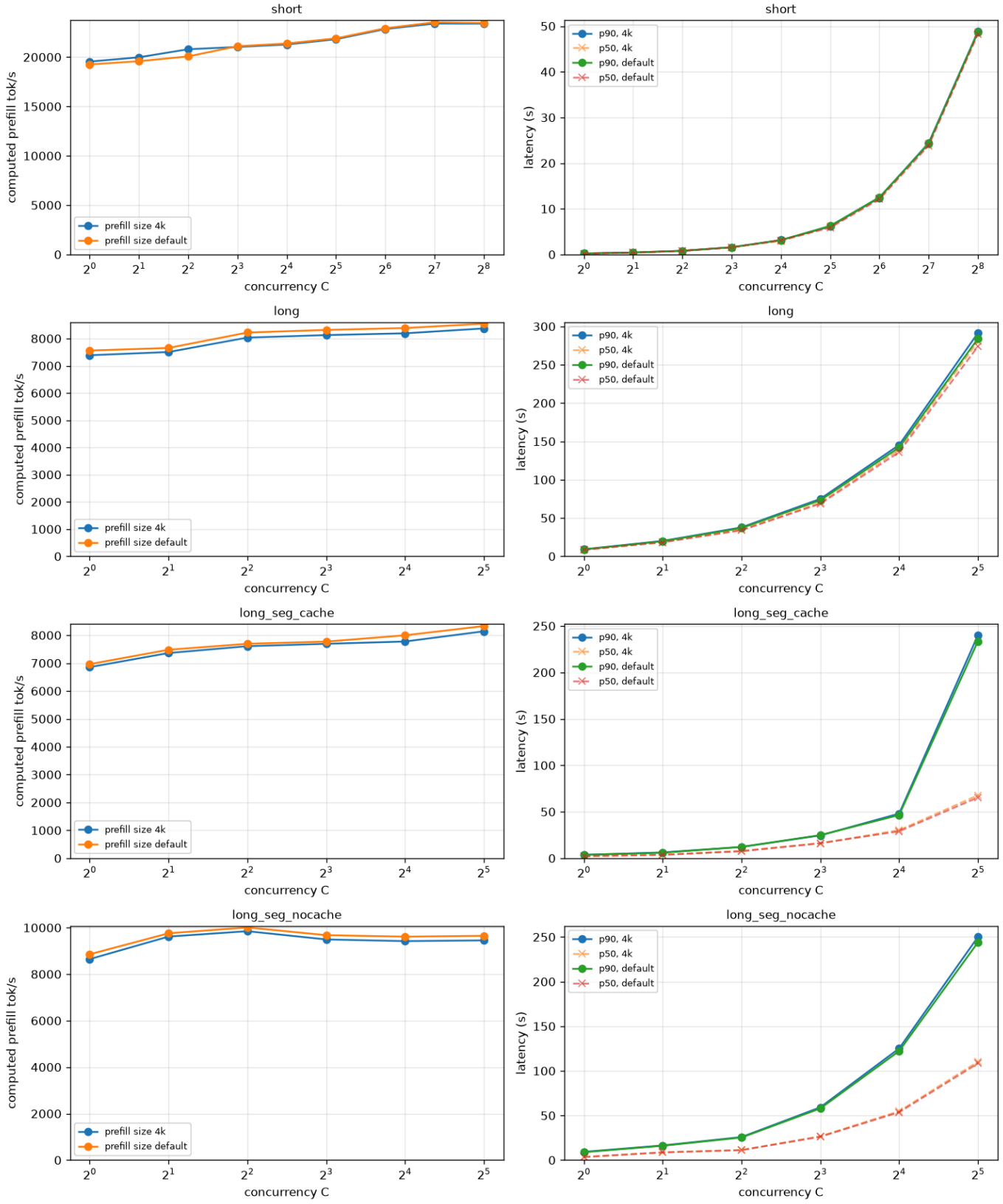
**short**（**~4k**）

| 并发 **C** | **4k** 计算 **tok/s** | **4k p50 / p90 (s)** | 默认 计算 **tok/s** | 默认 **p50 / p90 (s)** |
|---|---|---|---|---|
| 1 | 19,549 | 0.20 / 0.22 | 19,259 | 0.20 / 0.23 |
| 2 | 19,984 | 0.39 / 0.43 | 19,590 | 0.40 / 0.43 |
| 4 | 20,797 | 0.79 / 0.79 | 20,072 | 0.78 / 0.81 |
| 8 | 21,030 | 1.57 / 1.58 | 21,106 | 1.56 / 1.57 |
| 16 | 21,279 | 2.95 / 3.16 | 21,381 | 3.13 / 3.14 |
| 32 | 21,816 | 6.10 / 6.11 | 21,902 | 5.87 / 6.26 |
| 64 | 22,844 | 12.2 / 12.4 | 22,928 | 12.1 / 12.5 |
| 128 | 23,402 | 24.2 / 24.4 | 23,526 | 23.9 / 24.3 |
| 256 | 23,392 | 48.6 / 48.9 | 23,457 | 48.3 / 48.7 |

**long**（**~64k**）

| 并发 **C** | **4k** 计算 **tok/s** | **4k p50 / p90 (s)** | 默认 计算 **tok/s** | 默认 **p50 / p90 (s)** |
|---|---|---|---|---|
| 1 | 7,387 | 8.6 / 9.2 | 7,559 | 8.4 / 9.0 |
| 2 | 7,505 | 18.6 / 20.2 | 7,654 | 18.3 / 19.7 |
| 4 | 8,035 | 34.7 / 37.8 | 8,221 | 34.0 / 36.9 |
| 8 | 8,127 | 70.1 / 75.0 | 8,316 | 68.6 / 73.2 |
| 16 | 8,192 | 139 / 145 | 8,385 | 136 / 142 |
| 32 | 8,370 | 280 / 291 | 8,545 | 274 / 284 |

- 并发 1 时吞吐已经达到打满时的 83–88%，说明单个请求就把 **GPU** 的算力用掉了大半。剩下 10–20% 的提升来自 batch 更满、调度开销被摊薄。
- 吞吐一旦打满，延迟 ≈ C × 每个请求的 token 数 ÷ 吞吐，这就是右图里的线性增长：4k 请求为 3,968 ÷ 21–23k ≈ 0.18 秒 × C；64k 请求为 62.6k ÷ 8.3k ≈ 7.5 秒 × C，实测约 8.5 秒 × C。
- 64k 请求每个 token 比 4k 请求贵 2.75 倍（8.5k 对 23.4k tok/s），因为后面的 token 要对前面几万个 token 做 attention。




（左：计算 tok/s；右：延迟。横轴为并发数，log2 刻度。）

## 2. **prefill size**：**4k** 和默认值几乎没有差别

| 形状 | **4k** 打满 | 默认打满 | 差异 |
|---|---|---|---|
| short | 23,392 tok/s | 23,457 tok/s | +0.3% |
| long | 8,370 tok/s | 8,545 tok/s | +2.1% |
| long_seg_nocache（C=4） | 9,848 tok/s | 10,009 tok/s | +1.6% |
| long_seg_cache（C=16，服务 tok/s） | 20,209 tok/s | 20,777 tok/s | +2.8% |

4k token 的一次前向已经足够把 H20 的算力用满，再增大单次前向的 token 数，对吞吐几乎没有帮助。长请求在默认值下略快 2%，主要是切的块更少（64k 请求切 8 块，而不是 16 块）。

## 3. 给定延迟下的最大并发

在测过的并发档位里，p90 延迟不超过该值的最大 C。括号里是这个 C 下的计算 tok/s。`<1` 表示 C=1 也达不到。

| 形状 | **prefill size** | **p90 ≤ 0.25 s** | **≤ 0.5 s** | **≤ 1 s** | **≤ 2 s** |
|---|---|---|---|---|---|
| short | 4k | 1 (19,549) | 2 (19,984) | 4 (20,797) | 8 (21,030) |
| short | 默认 | 1 (19,259) | 2 (19,590) | 4 (20,072) | 8 (21,106) |

| 形状 | **prefill size** | **p90 ≤ 10 s** | **≤ 20 s** | **≤ 30 s** | **≤ 60 s** |
|---|---|---|---|---|---|
| long | 4k | 1 | 1 | 2 | 4 |
| long | 默认 | 1 | 2 | 2 | 4 |
| long_seg_cache（每段） | 4k | 2 | 4 | 8 | 16 |
| long_seg_cache（每段） | 默认 | 2 | 4 | 8 | 16 |
| long_seg_nocache（每段） | 4k | 1 | 2 | 4 | 8 |
| long_seg_nocache（每段） | 默认 | 1 | 2 | 4 | 8 |

因为吞吐几乎不随并发变化，可以用一个简单的公式外推没测过的档位：

> 最大并发 ≈ 延迟要求 × 打满时的吞吐 ÷ 每个请求的 **token** 数
>
> 例：4k 请求要求 p90 ≤ 5 秒 → 5 × 22,000 ÷ 3,968 ≈ 27 个并发（实测并发 16 时 p90 = 3.2 秒，并发 32 时 = 6.1 秒，吻合）。

分段形状的延迟是"每一段"的延迟，后面的段更长，所以 p90 主要由第 4 段决定。

## **4.** 分段 **+ KV cache**：能复用时服务量翻倍，**KV** 池装不下就失效

| 并发 **C**（轨迹数） | 不复用：服务 **tok/s** | 复用：服务 **tok/s** | 复用：计算 **tok/s** | 复用：缓存命中 | 不复用 **p90** | 复用 **p90** |
|---|---|---|---|---|---|---|
| 1 | 8,647 | 18,252 | 6,856 | 62.5% | 9.3 s | 3.7 s |
| 2 | 9,616 | 19,434 | 7,361 | 62.9% | 16.4 s | 6.3 s |
| 4 | 9,848 | 20,226 | 7,608 | 62.6% | 25.8 s | 12.1 s |
| 8 | 9,494 | 19,812 | 7,693 | 60.4% | 59.2 s | 24.6 s |
| 16 | 9,422 | 20,209 | 7,775 | 60.8% | 125 s | 48 s |
| 32 | 9,453 | **11,403** | 8,141 | **28.7%** | 250 s | 240 s |

（prefill size 4k。默认值那档的结果几乎一样：并发 1–16 时复用的服务 tok/s 为 18,658–20,777、命中 60–63%；并发 32 时服务 tok/s 跌到 11,686、命中 28.8%，p90 234 s。下面的分段明细也用 4k 那档的数据。）

每一段的延迟（**p50**）和缓存命中率：

| 并发 | 方式 | 第 **1** 段（**~16k**） | 第 **2** 段（**~32k**） | 第 **3** 段（**~48k**） | 第 **4** 段（**~64k**） |
|---|---|---|---|---|---|
| 1 | 复用 | 0.8 s，命中 30% | 2.0 s，命中 51% | 2.8 s，命中 65% | 3.6 s，命中 77% |
| 1 | 不复用 | 1.2 s | 3.2 s | 6.1 s | 9.7 s |
| 16 | 复用 | 12.7 s，命中 35% | 20.6 s，命中 50% | 35.2 s，命中 66% | 47.7 s，命中 75% |
| 16 | 不复用 | 17.1 s | 34.1 s | 70.5 s | 120 s |
| 32 | 复用 | 25.7 s，命中 35% | 43.0 s，命中 51% | 78.2 s，命中 **37%** | 226 s，命中 **9%** |
| 32 | 不复用 | 34.2 s | 65.5 s | 138 s | 235 s |

- 复用 **KV cache** 后，每一段只需要算新增的约 **16k token**，而不是整个前缀。所以服务 tok/s 翻倍，越靠后的段收益越大：第 4 段延迟从 9.7 秒降到 3.6 秒。
- 复用时计算 **tok/s** 反而更低（7.7k 对 9.5k）。因为新算的 token 要对前面已缓存的几万个 token 做 attention，平均每个新 token 更贵。看的是服务量而不是计算量的话，复用仍然划算得多。
- 第 **1** 段也有 **30–35%** 的命中：SWE 轨迹共用同一段 system prompt 和工具定义（约 5.7k token），会被其他轨迹缓存下来。
- **KV** 池放不下就失效：每条轨迹最后会占约 64k token 的 KV。并发 16 时 16 × 64k ≈ 100 万，刚好放进 107 万的 KV 池，命中率保持在 61%。并发 32 时需要约 200 万，超出了容量，前面轮次的 KV 被逐出：第 3、4 段的命中掉到 37% 和 9%，服务 tok/s 跌回 11.4k，几乎和不复用一样。 → 要靠 **KV** 复用来撑并发，在途轨迹数 × 每条轨迹的 **KV** 大小不能超过 **KV** 池。 这里只算到第 23 层，KV 池按 24 层分配，同样的显存能放的 token 数是 36 层时的 1.5 倍，这是只抽中间层的额外收益。

## **5.** 交叉检查：与完整模型对比

之前用 one_batch 跑完整 36 层模型（同样是 0.5.19、H20、单请求、不经过调度器）：2k–4k 请求约 13,420 tok/s，32k–64k 约 5,959 tok/s，64k–128k 约 4,212 tok/s。这次只算 24 层，4k 请求在并发 1 时为 19,549 tok/s（**1.46** 倍），64k 请求约 7,400–7,600 tok/s（约 **1.3–1.8** 倍，两边的长度分段不同）。理论上应该是 36 ÷ 24 = 1.5 倍，两者一致，说明截断确实生效了。两次测试的方式不同（one_batch 绕过调度器、这里经过调度器），只能作为量级上的核对。

## 局限

- 低并发档位的长请求样本少（例如 long C=1 只有 8 个请求，分段形状每一段只有 3–9 个），p90 只是大致值。高并发档位的样本都在 40 个以上。
- 延迟里包含了 SGLang tokenizer manager 和调度器的开销，但没有 HTTP；直接传 token ids，没有分词时间。
- `max_new_tokens=1`：每个请求 prefill 完会采样一个 token 再结束，采样器拿到的是全零 logits，开销可以忽略。
- 只测了 TP=1、单卡、这一种模型和层数。
- 用的是 SGLang 0.5.19（本机没有项目固定的 0.5.21）。

## 复现

```
bash run_serve_bench.sh shape.yaml
```

配置见仓库里的 `shape.yaml`（GPU、`latent_layer`、prefill size、并发档位、延迟要求）。数据处理见 `data/README.md`，测试流程见 README 的"请求形状 × 并发测试"一节。结果写到 `results/shape_<时间戳>/`：每个请求一行的 `ps<prefill size>/<形状>/c<C>.csv`、每档一行的 `summary.jsonl`、自动生成的 `serve_report.md` 和 `figs/serve.png`。本报告的数据来自 `results/shape_20261015_172145`（2026-10-15 17:21–20:49，GPU 7）。